

PaCT: Parallel Continuous Thought for Reasoning in Large Language Models

Anonymous Authors¹

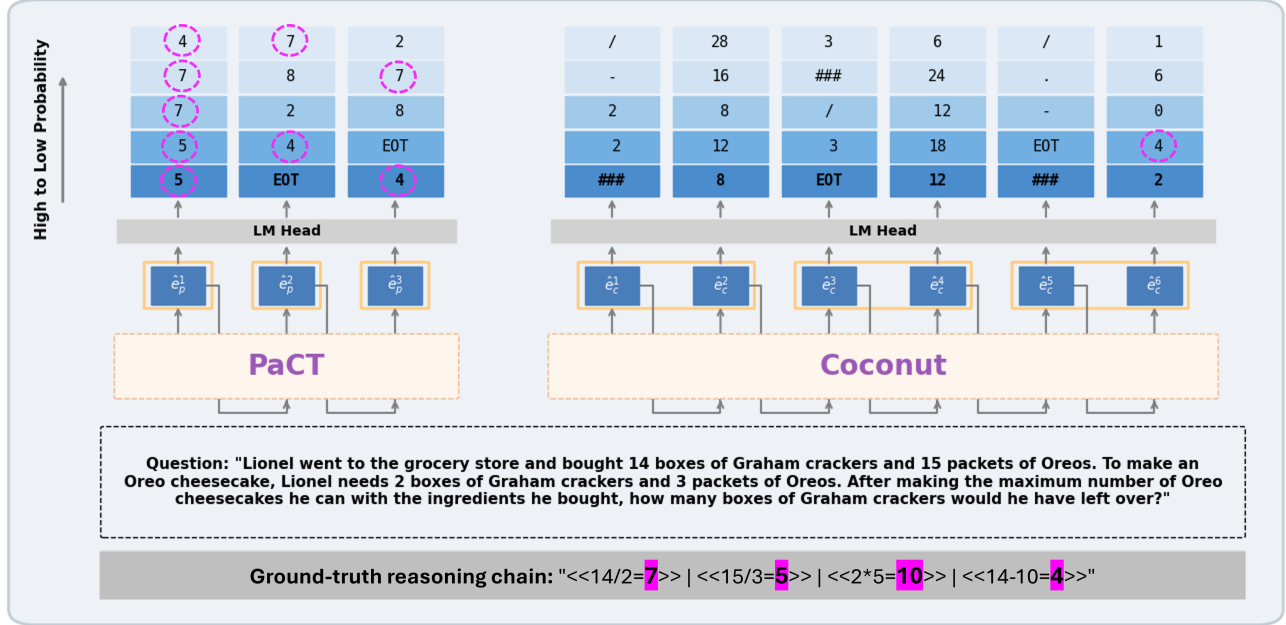


Figure 0: PaCT vs. Coconut: stage-wise latent resynthesis without recurrent unrolling. Coconut generates continuous thought by recurrently propagating latent states, inducing RNN-like dependencies. PaCT instead re-synthesizes latent reasoning states in parallel from the current context at each stage and conditions causal decoding on them, decoupling reasoning depth from serial computation. This yields more information-rich latent representations, achieving strong reasoning with fewer latent states, and enables 4–9× faster training with comparable or improved accuracy.

Abstract

Large language models (LLMs) typically reason in the language space, expressing intermediate computation as chains of thought (CoT). While effective, this ties reasoning to sequential text token generation, where many tokens primarily ensure textual coherence without directly contributing to the underlying reasoning process, and the cost of reasoning scales with the length of the generated trace. Recent works explore continuous latent reasoning by replacing portions of CoT with internal states by relying on recurrent latent unrolling, introducing long serial dependencies inside the Transformer, and limiting hardware parallelism. We introduce *Parallel Continuous Thought* (PaCT), which eliminates recurrent unrolling by organizing latent reasoning into stages. At each stage, PaCT synthesizes a set of latent reasoning states *in parallel* using a learnable query codebook and cross-attention to the current context, refines them via bidirectional interaction within the stage, and conditions standard

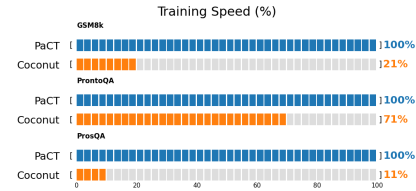


Figure 1. PaCT vs. Coconut: training speed comparison.

causal decoding through a strictly causal interface. PaCT challenges the assumption that continuous latent reasoning must rely on latent state propagation, showing that reasoning depth can instead be achieved through repeated, context-grounded latent resynthesis. Empirically, PaCT trains substantially faster than recurrent latent unrolling while matching or improving reasoning performance across diverse settings and model scales. The analyses demonstrate that the stages contribute meaningfully to the final predictions, encode distinct information across stages, and stabilize the internal dynamics earlier during reasoning. Our code and model weights are available at [PaCT](#).

1. Introduction

Large language models (LLMs) have recently demonstrated strong multi-step reasoning capabilities when prompted to generate explicit chains of thought (CoT), in which intermediate reasoning steps are expressed as natural language tokens prior to producing a final answer (?). While effective, this paradigm tightly couples reasoning to the discrete token space of language, forcing models to externalize internal computation through sequential text generation. Language tokens, however, are optimized for communication rather than for the core requirements of reasoning such as abstraction, conceptualization, and computation (?????). Moreover, autoregressive decoding commits to a single partial trajectory token by token, and the computational cost of reasoning scales linearly with the number of generated tokens (?). These constraints make detailed reasoning both expensive and inherently sequential.

A substantial body of work has sought to alleviate these issues through improved prompting strategies and decoding-time search. Techniques such as decomposed prompting (?), least-to-most prompting (?), self-consistency and guided beam-style decoding (?), and tree-based search over reasoning trajectories (?) improve robustness by exploring or aggregating multiple textual reasoning paths. Related approaches introduce test-time deliberation signals that encourage models to “think” before committing to an answer (?). Despite their success, these methods operate entirely in token space and therefore inherit its fundamental limitation: each reasoning step must still be generated, evaluated, and committed sequentially. As a result, they improve exploration within the same substrate rather than enabling more abstract or information-dense internal computation.

Recent work has therefore explored shifting reasoning from explicit language tokens into the model’s continuous hidden state space. *Chain of Continuous Thought* (Coconut) introduced the idea of replacing segments of the textual chain-of-thought with continuous latent representations that are never decoded as language and are trained implicitly through their effect on final answer likelihood (?). Empirically, Coconut demonstrates that models trained in this manner can exhibit emergent search behaviors resembling breadth-first exploration while requiring fewer explicit reasoning tokens. Complementing this empirical evidence, theoretical analyses show that continuous latent representations can be strictly more expressive than discrete token sequences: *Reasoning by Superposition* proves that a single latent step can encode superpositions of multiple candidate reasoning states that would require sequential exploration in token space (?). Together, these results establish continuous latent reasoning as a promising alternative substrate for complex reasoning.

Despite this promise, existing methods of continuous latent

reasoning suffer from a critical structural limitation. In Coconut, continuous thoughts are generated *sequentially*: each latent vector is produced by feeding the previous hidden state back as the next input embedding (?). This design introduces a recurrent unrolling within the Transformer, reinstating an RNN-like dependency structure and causing the critical path length to grow linearly with the number of latent steps. Consequently, training becomes substantially slower, hardware parallelism is poorly utilized, and optimization stability degrades as latent depth or model size increases, as gradients must propagate through a long sequence of dependent latent states, causing compounding of early representational instabilities.

Importantly, this recurrent structure imposes not only a computational bottleneck but also a representational one. When latent thoughts are generated by recursively transforming prior latent states, each latent must accumulate and carry forward information from earlier steps, as future computation has access only to the most recent latent. The absence of an explicit stage boundary encourages successive latents to entangle multiple reasoning roles within a single evolving representation, rather than forming distinct, stage-specific abstractions. Moreover, because learning signals are applied only through the final answer likelihood, recurrent unrolling biases the model toward encoding global summaries early rather than organizing reasoning into separable stages. In contrast, reasoning architectures that re-synthesize latent representations from the current context at each stage can refactor information anew, discouraging unstructured accumulation and enabling specialization across stages. We examine these representational effects empirically through detailed analyses later in the paper.

Several recent methods aim to improve the efficiency of latent reasoning but do not address this architectural bottleneck. CoLaR dynamically compresses chains of thought into fewer latent steps and optimizes reasoning length via reinforcement learning, but it retains autoregressive latent generation and explicitly supervises latents to preserve token-level semantics (?). CODI similarly distills explicit CoT into continuous latent representations through self-distillation, treating latents as compact surrogates for textual reasoning traces (?). Token Assorted mixes latent and text tokens during reasoning but likewise preserves sequential latent computation (?). These approaches reduce the number of latent steps or compress existing reasoning traces, yet they do not remove the core sequential dependency induced by recurrent latent unrolling, nor do they enable latent representations to reorganize computation across reasoning stages.

In this work, we introduce *Parallel Continuous Thought* (PaCT), a reformulation of continuous latent reasoning that eliminates recurrent latent unrolling altogether. *PaCT challenges the implicit assumption that continuous latent reason-*

ing must rely on latent state propagation across steps, showing instead that reasoning depth can be achieved through repeated, context-grounded latent resynthesis by applying parallelism. Instead of generating latent thoughts sequentially, PaCT synthesizes a set of latent representations *in parallel* at each reasoning stage using a learnable codebook of query vectors and cross-attention to the current textual context. These latents are then refined through bidirectional interaction within a latent representation set before conditioning subsequent decoding. By re-synthesizing latents at each stage rather than recursively accumulating prior latent states, PaCT decouples reasoning capacity from serial computation depth, enables efficient hardware parallelism, and significantly reduces training and optimization overhead, while encouraging stage-wise specialization of latent representations and preserving standard causal decoding.

Empirically, PaCT matches or exceeds the reasoning accuracy of Coconut across diverse mathematical and logical reasoning benchmarks and model scales, while achieving 4–9 $\times$ reductions in training time (Figure ??). Beyond efficiency, our analyses show that PaCT produces information-rich latent reasoning where fewer latent states suffice to achieve strong performance, increasing reasoning depth yields consistent gains without increasing latent width, and successive stages encode distinct, context-grounded information rather than redundantly accumulating latent history. Together, these results demonstrate that parallel latent synthesis is not merely an acceleration strategy, but an architectural improvement that enables both scalable and higher-quality continuous latent reasoning. Our main contributions are -

(1) Stage-wise latent resynthesis without recurrent unrolling. We eliminate recurrent latent state propagation by re-synthesizing latent reasoning states at each stage from the current context, decoupling reasoning depth from serial computation. This enables parallel latent synthesis via cross-attention and yields substantially faster training while matching or improving reasoning accuracy (Figure ??).

(2) Information-rich latents via within-stage de-liberation. We introduce bidirectional interaction among latent states within each reasoning stage, producing more information-dense latent representations and improving reasoning capacity without increasing latent width (Figure ??).

2. Methodology

2.1. Objective and problem formulation

We study supervised reasoning tasks where each training example consists of a question q , an optional chain-of-thought expressed as text tokens $r = (r_1, \dots, r_M)$, and a final an-

swer $a = (a_1, \dots, a_N)$. The full textual sequence is

$$y = [q; r; a].$$

A standard causal language model (LM) parameterized by θ learns $p_\theta(y)$ via maximum likelihood training.

Goal. Our goal is to enable multi-step reasoning using *continuous* internal representations while preserving standard autoregressive decoding of the answer, and crucially, avoiding the *serial* compute bottleneck that arises when continuous reasoning is implemented through recurrent latent unrolling.

Key bottleneck. Chain of Continuous Thought (Coconut) (?) replaces segments of textual chain-of-thought (CoT) tokens with continuous latent states, but generates these states *sequentially* by feeding the previous hidden state back as the next input embedding. This induces recurrent dependencies across latent steps, causing serial depth to grow linearly with the number of latent states, limiting hardware parallelism, and reducing representational separation across reasoning stages.

Hypothesis. We hypothesize that the benefits of continuous latent reasoning do not require recurrent latent unrolling. Instead, effective reasoning can be achieved by (i) synthesizing a *set* of latent reasoning states in parallel from the current textual context, (ii) allowing structured interaction *within* each reasoning stage, and (iii) conditioning autoregressive decoding on these states through a causal interface, without directly propagating latent states across stages.

2.2. Preliminaries: causal LMs and Coconut-style latent replacement

A causal Transformer LM maps token prefixes $x_{\leq t}$ to hidden states $H_t \in \mathbb{R}^{t \times d}$ and predicts the next token via

$$p_\theta(x_{t+1} \mid x_{\leq t}) = \text{softmax}(Wh_t).$$

Coconut (?) replaces a prefix of textual CoT tokens with continuous latent states that are not decoded as language. During this latent phase, each latent state is generated by feeding the previous hidden state as the next input embedding, creating a recurrent unrolling within the Transformer, reinstating an RNN-like bottleneck within an otherwise parallel architecture.

2.3. Parallel Continuous Thought (PaCT)

PaCT replaces recurrent latent unrolling with *stage-wise parallel latent reasoning*. Rather than propagating latent states across time, PaCT organizes reasoning into a sequence of stages. Each stage constructs a fresh set of latent reasoning states directly from the current textual context, allows structured interaction among them, and then exposes them

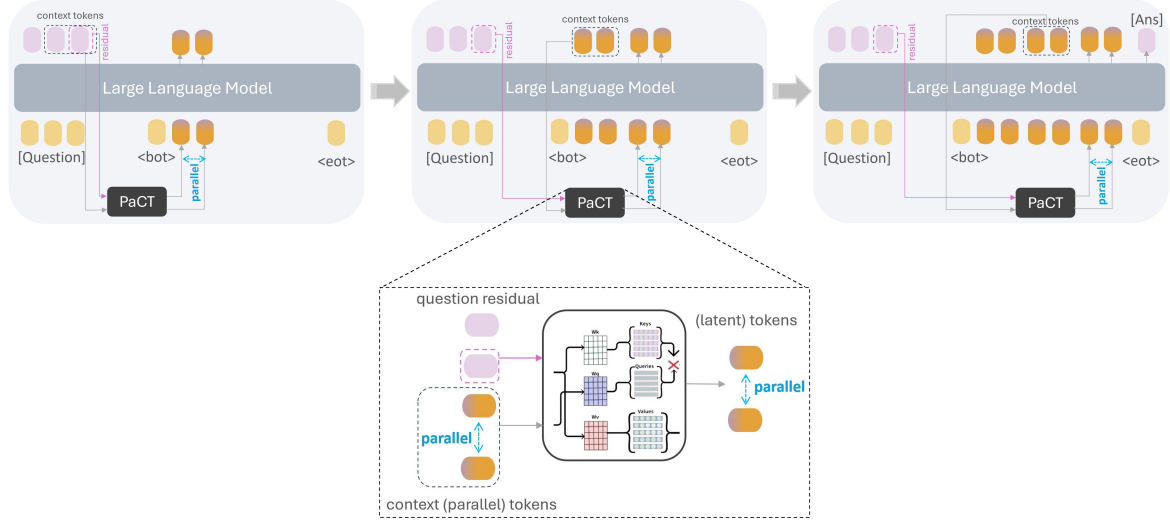
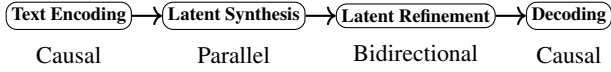


Figure 2. **Parallel Continuous Thought (PaCT)**. At each reasoning stage, a set of latent reasoning states is synthesized in parallel from the current textual context, refined through bidirectional interaction, and used to condition subsequent autoregressive decoding.

to subsequent decoding. Importantly, latent states do not directly transform or depend on latent states from earlier stages. Parallelism in PaCT arises as a consequence of stage-wise latent resynthesis: since latent reasoning states are re-synthesized from the current context rather than propagated from previous states, they can be generated simultaneously within each stage.

Conceptually, a single PaCT reasoning stage proceeds as follows:



This structure preserves causal text processing and decoding, while enabling parallel computation and bidirectional interaction *within* each reasoning stage.

2.3.1. LATENT CAPACITY VIA LEARNABLE QUERY SLOTS

Let L denote the number of latent reasoning states per stage. PaCT maintains a learnable set of query vectors

$$Q = \{q_1, \dots, q_L\}, \quad q_i \in \mathbb{R}^d,$$

which define the *width* of the latent reasoning set. These queries persist across examples and extract task-relevant summaries from the current textual context. We use the term *latent reasoning state* to denote a continuous hidden vector that participates in reasoning but is never decoded as language.

2.3.2. PARALLEL LATENT SYNTHESIS

Given the hidden states $H \in \mathbb{R}^{T \times d}$ of the current text prefix, PaCT synthesizes all latent reasoning states for a stage in a

single cross-attention operation:

$$Z = \text{CrossAttn}(Q, K, V), \quad K = V = H.$$

This produces $Z \in \mathbb{R}^{L \times d}$ in parallel. In contrast to Coconut, the number of latent reasoning states is no longer tied to the number of sequential Transformer invocations, and no latent state depends on another for its construction.

2.3.3. LATENT REFINEMENT WITHIN A STAGE

After synthesis, latent reasoning states interact *only within the same stage* through latent-only self-attention with a bidirectional mask:

$$Z' = \text{SelfAttn}(Z).$$

This bidirectional interaction enables coordination among latent hypotheses within a stage. It does not violate causality, as latent states do not attend to future answer tokens, and text tokens do not attend to latent states. No bidirectional interaction occurs across different stages.

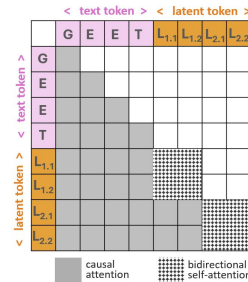


Figure 3. Bidirectional interaction among latent reasoning states within a single stage, while preserving causal text decoding.

2.3.4. CONDITIONING DECODING ON LATENT REASONING STATES

Autoregressive decoding is conditioned on latent reasoning states through a causal interface. Answer tokens may attend to both prior text tokens and all latent reasoning states produced in *earlier stages*. Latent states themselves never attend to answer tokens.

Concretely, we concatenate latent reasoning states into the attention context and add a cross-attention pathway from text representations to latent keys and values. This realizes the factorization

$$p(a | q) \approx p(a | q, Z^{(1:S)}),$$

where $Z^{(1:S)}$ denotes the latent reasoning states across S stages. The attention masking rules that enforce this causal structure are detailed in Appendix ??.

2.3.5. MULTI-STAGE LATENT REASONING

PaCT applies the above procedure for S reasoning stages. All latent states within a stage are synthesized simultaneously from a restricted window of the most recent k positions in the sequence, which may include text tokens, latent states from earlier stages, or both. The window size k is a fixed hyperparameter, with its impact analyzed in Table ?? and Figure ?. This yields two independent scaling axes: width L (latent states per stage) and depth S (number of reasoning stages). Increasing depth adds reasoning capacity without increasing latent width or serial dependency.

2.4. Training

PaCT is trained using a curriculum similar to Coconut (?), which progressively replaces steps of CoT with continuous latent reasoning tokens. At curriculum stage s , a prefix of the textual chain-of-thought is replaced by s latent reasoning stages, each consisting of L latent reasoning states synthesized in parallel. The remaining CoT tokens (if any) and the answer remain as language tokens. This curriculum stabilizes optimization compared to training directly from (q, a) pairs.

Training minimizes the standard next-token prediction loss on text tokens, with the loss masked on latent positions:

$$\mathcal{L}(\theta) = - \sum_{t \in \mathcal{I}_{\text{text}}} \log p_{\theta}(x_t | x_{<t}, Z^{(1:S)}).$$

The full training algorithm is provided in Appendix ??.

2.5. Computational analysis

Efficiency is governed by serial depth, i.e., the length of the critical path that cannot be parallelized. Let T be the number of text decoding steps and n_{lat} the total number of latent reasoning states.

In Coconut, latent reasoning states are generated recurrently, yielding

$$D_{\text{seq}}^{\text{Coconut}} = T + n_{\text{lat}}.$$

In PaCT, L latent reasoning states are generated in parallel per stage, and only the number of stages S contributes to serial depth:

$$D_{\text{seq}}^{\text{PaCT}} = T + S, \quad n_{\text{lat}} = SL.$$

Thus, for a fixed latent budget, PaCT reduces latent-induced serial depth by a factor of L . PaCT shifts computation from the critical path to parallel attention and matrix multiplications, which modern accelerators can efficiently amortize.

3. Results and Discussion

Results overview. PaCT removes the recurrent latent-unrolling bottleneck while preserving (and often improving) reasoning performance. Across GSM8k, ProntoQA, and ProsQA, PaCT matches or exceeds Coconut while reducing training time by $3.2\text{--}4.7\times$ (Table ??), and these gains persist at larger model scales (Table ??). We then analyze *why* PaCT works: (i) each latent stage contributes measurably to the final prediction (Fig. ??); (ii) PaCT promotes stage-wise specialization and scales well with depth (Table ??, Fig. ??); (iii) latent trajectories exhibit interpretable confidence dynamics (Table ??, Fig. ??); and (iv) PaCT stabilizes internal dynamics relative to Coconut (Fig. ??). Finally, we validate a key architectural choice of restricting the cross-attention context for latent synthesis (Sec. ??).

3.1. Experimental Setup

To ensure a fair, direct comparison, we follow Coconut (?), matching model architectures, curriculum schedules, datasets, and optimization settings. This isolates the effect of PaCT’s parallel latent reasoning from other confounders.

Base models. Our primary experiments use GPT-2 Small (124M) as the base causal LM, without modifying the underlying Transformer layers. To evaluate scaling with model size, we additionally run experiments using LLaMA-3B and LLaMA-8B as base models (Table ??). In all cases, PaCT is integrated on top of the base model with the same module design.

PaCT configuration. Unless stated otherwise, PaCT uses a fixed configuration: a latent query/codebook projection dimension of 128 and 4 attention heads for cross-attention. Latent synthesis uses a restricted context window consisting of the two most recent hidden states; we ablate this choice in Sec. ??.

Curriculum training. We use Coconut’s multi-stage curriculum, progressively replacing textual CoT tokens with latent reasoning stages. For GSM8k, we first fine-tune with

CoT supervision for 25 epochs, followed by 25 epochs of curriculum-based latent training. For ProntoQA and ProsQA, models are trained for 50 epochs under the same curriculum schedule as Coconut. We set the maximum number of latent stages to 3 for GSM8k and 6 for ProntoQA and ProsQA.

Optimization. We use AdamW with learning rate 1×10^{-4} , weight decay 0.01, and effective batch size 128. We evaluate on GSM8k (math), ProntoQA (deductive logic), and ProsQA (proof-oriented reasoning). Following prior work, the continuous thought parameter is set to $c_{\text{thought}} = 2$ for GSM8k and $c_{\text{thought}} = 1$ for ProntoQA and ProsQA. All experiments are conducted on a single node of H100 with four 80 GPUs.

3.2. Parallel latent reasoning improves training efficiency without sacrificing accuracy

Table ?? shows that PaCT consistently reduces training time while maintaining strong reasoning accuracy. On GSM8k, PaCT improves accuracy over Coconut (34.4 vs. 32.0) and reduces training time from 15.40 to 3.30 hours (4.7 $\times$). On ProntoQA, PaCT matches Coconut’s accuracy (100.0 vs. 99.8) while reducing training time from 2.11 to 0.65 hours (3.2 $\times$). On ProsQA, PaCT improves accuracy (98.67 vs. 97.0) and reduces training time from 6.42 to 1.58 hours (4.1 $\times$). These gains align with the serial-depth analysis in Sec. ??: PaCT removes recurrent latent unrolling and shifts latent computation to parallel attention operations within each stage. Detailed inference analysis is present in appendix ??.

3.3. Scaling to larger model sizes

Table ?? shows that PaCT’s efficiency gains persist at larger model scales. On the 3B model, PaCT reduces training time from 51.5 to 11.02 hours (4.7 $\times$) while improving accuracy (31.7 $\rightarrow$ 36.4). On the 8B model, PaCT reduces training time from 105.9 to 22.68 hours (4.7 $\times$) while matching Coconut’s accuracy (43.6). This consistency supports the core claim that eliminating recurrent latent unrolling reduces critical-path length independent of model size.

3.4. Every latent stage contributes to performance

Efficiency gains would be unconvincing if latent stages acted as inert compute. We therefore test whether each reasoning stage contributes to the final prediction by ablating entire stages at inference time. Concretely, we progressively remove latent stages (each stage contains $L=2$ latent reasoning states on GSM8k) and measure the resulting change in log-probability assigned to (i) the model’s generated answer token and (ii) the ground-truth answer token. Figure ?? shows a clear monotonic trend: retaining more stages yields higher answer likelihood, and removing later stages causes

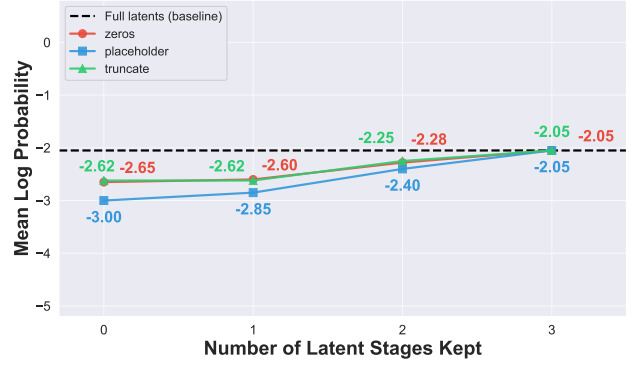


Figure 4. Impact of latent stage ablation on answer confidence. We measure the mean log-probability of ground-truth answers on GSM8k while progressively removing latent stages via zeroing, placeholder replacement, or truncation. The observed monotonic increase in probability as more stages are retained demonstrates that PaCT utilizes each computational stage to actively refine its final prediction.

the largest confidence drops. This indicates that PaCT stages are actively used to refine the final prediction.

3.5. Latent stage specialization and depth scaling

PaCT uses *stage-wise resynthesis*: each stage constructs its latent states directly from the current textual context and previously generated latent states, rather than recurrently transforming a single latent trajectory as in Coconut. Table ?? offers a mechanistic view of this difference. In PaCT, latent states within the same stage are nearly identical (0.99 cosine similarity), while cross-stage similarity is much lower (0.31), suggesting that successive stages encode distinct information. Coconut exhibits the reverse: within-stage latent states are diverse (0.30), but cross-stage similarity is higher (0.63), consistent with recurrent unrolling that encourages information to be carried forward along a single evolving state. This structure has a direct implication: if within-stage latent states are largely redundant in PaCT, compute is better allocated toward *depth* (more stages) than toward *width* (more latent states per stage), provided a minimum of within-stage width is preserved to sustain the bidirectional refinement that makes each stage informative. We validate this first in Figure ??, which shows consistent gains from increasing the number of stages across datasets, and then in Table ??, which fixes the total latent budget and sweeps over width–depth splits.

Width-depth tradeoff under a fixed latent budget. To disentangle the contributions of within-stage width L and reasoning depth S , we fix the total latent budget at $L \cdot S = 6$ on GSM8k and sweep over the four integer factorizations, keeping every other hyperparameter at the default configuration. Table ?? reports the resulting accuracies.

The sweep reveals a clearly non-monotonic pattern. Placing the entire budget in width with a single reasoning

Method	GSM8k		ProntoQA		ProsQA	
	Acc. (%)	Train Time (hrs)	Acc. (%)	Train Time (hrs)	Acc. (%)	Train Time (hrs)
CoT	42.90 $\pm$ 0.2	2.90	98.80 $\pm$ 0.8	0.55	77.50 $\pm$ 1.9	0.91
No-CoT	16.50 $\pm$ 0.5	2.18	93.80 $\pm$ 0.7	0.41	76.70 $\pm$ 1.0	0.68
Pause Token	16.40 $\pm$ 1.8	2.47	77.70 $\pm$ 21.0	0.47	75.90 $\pm$ 0.7	0.77
iCoT	30.00	2.47	99.80 $\pm$ 0.3	0.47	98.20 $\pm$ 0.3	0.77
Coconut	32.00 $\pm$ 1.5	15.40	99.80 $\pm$ 0.2	2.11	97.00 $\pm$ 0.3	6.42
CoLaR	26.40 $\pm$ 1.7	3.10	96.60 $\pm$ 0.6	0.45	75.00 $\pm$ 0.8	6.21
CODI	33.81 $\pm$ 1.2	5.90	78.90 $\pm$ 0.6	0.89	*	*
PaCT	34.40 $\pm$ 0.1	3.30	100.0 $\pm$ 0.1	0.65	98.67 $\pm$ 0.2	1.58

Table 1. **Accuracy and training efficiency across reasoning methods.** Results are reported on GSM8k, ProntoQA, and ProsQA. PaCT matches or exceeds Coconut across all benchmarks while substantially reducing training time. Results for Coconut, CoLaR, and CODI are obtained by rerunning their released implementations under our experimental setup. *CODI’s training was not stable on ProsQA, hence results have been omitted.

Table 2. **Scaling with model size.** Accuracy and training time across larger base models on GSM8k. Training time in hrs.

Model	Params	Coconut		PaCT	
		Acc	Time	Acc	Time
LLaMA 3.2	3B	31.7	51.5	36.4	11.02 (5x)
LLaMA 3	8B	43.6	105.9	43.6	22.68 (5x)

Table 3. **Latent similarity diagnostics.** PaCT exhibits high within-stage similarity but low cross-stage similarity, while Coconut exhibits the opposite trend.

Metric	PaCT	Coconut
Within-stage cosine similarity	0.99 $\pm$ 0.01	0.30 $\pm$ 0.17
Cross-stage cosine similarity	0.31 $\pm$ 0.21	0.63 $\pm$ 0.14

stage ($L=6$, $S=1$) performs worst (26.0%), confirming that PaCT’s gains are not driven by within-stage latent quantity: beyond a small L , additional within-stage states are redundant. Depth therefore matters — all multi-stage configurations outperform $S=1$. However, collapsing width entirely to a single latent per stage ($L=1$, $S=6$) also degrades performance (29.6%), because the bidirectional refinement introduced in Sec. ?? requires at least two within-stage latents to act on. The balanced allocation ($L=2$, $S=3$, matching our default in Table ??) attains the best accuracy (34.4%). The broader conclusion is that depth dominates width, but a minimal amount of within-stage width ($L \geq 2$) is necessary to realise the benefits of stage-wise bidirectional refinement — consistent with the within-stage similarity structure in Table ?? and with the submanifold structure documented later in Sec. ??.

3.6. Latent token decoding analysis

Although PaCT latent states are not trained to be decoded as language, we can obtain a coarse qualitative probe by projecting each latent embedding into vocabulary space and inspecting the top predicted token at that latent position after the Transformer pass. This probe is not meant to

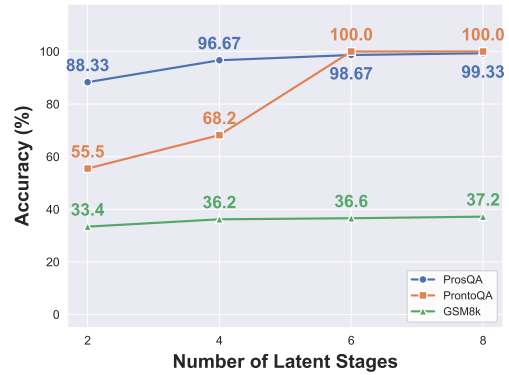


Figure 5. **Accuracy vs. number of stages.** Increasing the number of latent stages improves accuracy across benchmarks.

Table 4. **Width–depth allocation under a fixed latent budget.** Holding the total number of latent reasoning states constant at $L \cdot S = 6$ on GSM8k, we vary how the budget is split between within-stage width L and reasoning depth S . A balanced allocation that favours depth ($L=2$, $S=3$) is best; both pure width ($S=1$) and pure depth ($L=1$) underperform.

Width L	Depth S	Accuracy (%)
6	1	26.0
3	2	30.0
2	3	34.4
1	6	29.6

claim that latents are “natural language”; rather, it helps visualize whether the model’s internal trajectory moves from intermediate quantities toward the final answer.

Table ?? shows that the final latent’s top-1 probability separates correct from incorrect predictions, indicating that latent trajectories carry a meaningful confidence signal. Figure ?? illustrates two representative cases: in correct predictions, decoded latents progress from intermediate quantities to the final answer with sharpening confidence, while in incorrect predictions the model identifies partial intermediates but

Table 5. Final-latent confidence for correct vs. incorrect predictions. Top-1 probability at the final latent position.

Model	Correct	Incorrect
PaCT	0.680 ± 0.315	0.257 ± 0.223
Coconut	0.744 ± 0.244	0.312 ± 0.232

Example 1

Correct

Question: Kenny played basketball last week. He ran for twice as long as he played basketball, and he practiced on the trumpet for twice as long as he ran. If he practiced on the trumpet for 40 hours, how many hours did Kenny play basketball?

Ground Truth CoT: $40 \div 2 = 20 \rightarrow 20 \div 2 = 10$

Predicted: 10 **GT:** 10

	L0	L1	L2	L3	L4	L5
Token	20	20	10	10	10	10
Prob	4.7%	6.8%	15%	31%	95%	94%

Interpretation: Latents capture the intermediate (20) then converge to the final answer (10) with sharp confidence increases.

Figure 6. **Representative decoded latent trajectories.** Within-stage latent pairs often decode to identical tokens, consistent with high within-stage similarity in Table ?? . Additional examples are provided in Appendix ?? .

fails to compose them correctly. Additional examples and related analyses comparing single-latent PaCT to double-latent Coconut are provided in ?? and ?? respectively. An algorithm of the probing process can be found in ?? .

3.7. PaCT promotes stable latent dynamics

Beyond accuracy and efficiency, we examine how PaCT alters internal reasoning dynamics. We analyze hidden representations using the Information Contribution to the Residual Stream (ICR) score (?), which measures deviation of residual updates from attention-based explanations. Figure ?? shows consistently lower ICR for PaCT in the middle layers, indicating earlier stabilization relative to Coconut. This aligns with PaCT’s stage-wise latent resynthesis, where each stage is grounded in the current context rather than accumulating latent history through recurrent unrolling.

3.8. Effect of context length for latent synthesis

A key design choice in PaCT is the number of context tokens used as keys and values for cross-attention during latent synthesis. Table ?? shows that expanding this context beyond a small window does not improve performance and can even degrade it. Figure ?? explains this behavior: regardless of available context length, cross-attention concentrates on the most recent hidden states.

This aligns with prior observations that Transformer atten-

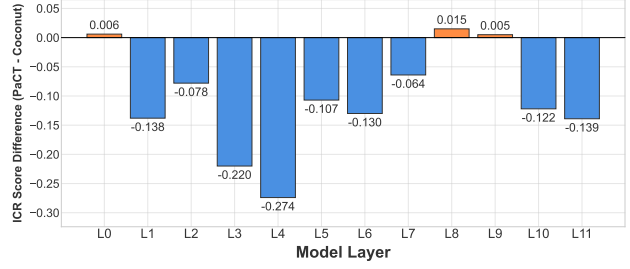


Figure 7. **Layer-wise difference in ICR scores between PaCT and Coconut.** Negative values indicate lower ICR for PaCT. Table 6. Effect of number of context tokens on PaCT (GPT-2) performance on GSM8k.

Number of context tokens	Accuracy
2	34.4
4	33.0
8	32.3
All	30.6

tion exhibits a strong inductive bias toward recency (?). In PaCT, this bias is amplified because latent synthesis attends not only to text tokens but also to latent states from previous stages, which already summarize the cumulative context. As a result, attending to recent latents provides access to relevant historical information without requiring long text-level context.

Together, these results justify a restricted context window for latent synthesis, which aligns with the model’s learned attention bias while improving computational efficiency.

3.9. Perturbing PaCT’s latent states

Having established that each reasoning stage contributes to PaCT’s final prediction (Sec. ??) and that latent trajectories carry a meaningful confidence signal (Sec. ??), we now probe what the latent reasoning states actually encode by intervening on them directly at inference time. The guiding hypothesis is simple: if the latents carry task-relevant reasoning information, destructive perturbations that erase or replace them should reduce accuracy substantially, whereas structure-preserving noise should be largely absorbed.

Probes. We apply five inference-time interventions to a trained PaCT model on GSM8k, without any additional training: (i) *zeroing*, where every latent state is replaced with the zero vector; (ii) *random-norm replacement*, where each latent is replaced with a Gaussian random vector rescaled to match the empirical latent norm; (iii) *cross-example swap*, where latents synthesized for a randomly paired, unrelated example are substituted for the current ones; (iv) *no-prompt decode*, where answer tokens are prevented from attending back to the question prefix during decoding so that the answer must be reconstructed from the latent states alone;

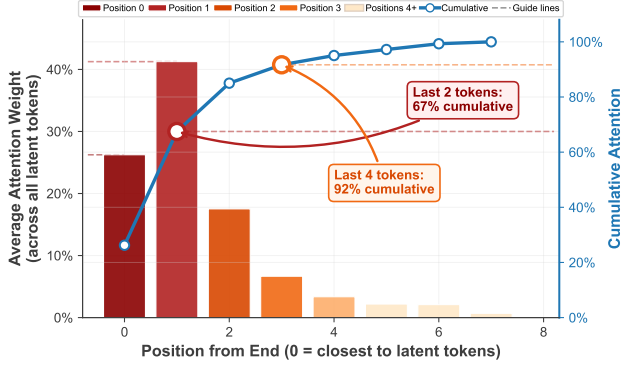


Figure 8. Cross-attention weight distribution across different number of context tokens. Position 0 denotes the last token before latent synthesis, position 1 the second-to-last, and so on. The model concentrates attention on the most recent 1–2 positions regardless of available context.

Table 7. Perturbing PaCT’s latent reasoning states on GSM8k. Destructive interventions (zeroing, random-norm replacement, cross-example swap) collapse accuracy, while structure-preserving Gaussian noise is largely absorbed. Restricting answer decoding to attend only to latent states (no-prompt decode) also collapses accuracy, indicating that the latents encode information not redundantly present in the prompt.

Condition	Accuracy (%)
PaCT (clean)	34.40
Zero all latents	7.80
Random vectors, matched norm	8.77
Swap latents across examples	4.27
No-prompt decode (answer attends only to latents)	7.40
Gaussian noise, $\sigma=0.01$	33.62
Gaussian noise, $\sigma=0.05$	33.62
Gaussian noise, $\sigma=0.10$	33.78
Gaussian noise, $\sigma=1.00$	31.77

and (v) *additive Gaussian noise* $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$ added to every latent, swept over $\sigma \in \{0.01, 0.05, 0.1, 1.0\}$.

Table ?? summarizes the results. Three observations follow. *First, destructive perturbations collapse performance.* Zeroing (7.80%), random-norm replacement (8.77%), and cross-example swap (4.27%) each reduce accuracy by more than 25 absolute points from the 34.40% clean baseline, so the latent states are load-bearing for PaCT’s predictions rather than acting as passive padding. *Second, the latent pathway carries non-redundant information.* When answer decoding is restricted to attend only to the latent states, accuracy falls to 7.40%, showing that the latents encode signal that the prompt path alone cannot reconstruct. *Third, latents occupy a structured submanifold.* Gaussian perturbations stay close to the clean baseline across the entire sweep (33.62% at $\sigma=0.01$, 33.62% at $\sigma=0.05$, 33.78% at $\sigma=0.1$, and 31.77% even at $\sigma=1.0$), whereas random unit-norm replacements — which share the noise’s isotropy but destroy the learned directions — collapse it (8.77%). The latents

Table 8. Out-of-distribution generalization. All models are trained on GSM8k and evaluated zero-shot on three held-out math benchmarks. PaCT matches or exceeds Coconut on every split.

Dataset	PaCT (%)	Coconut (%)
GSM-HARD	7.88	7.90
MultiArith	82.34	82.00
SVAMP	37.67	36.40

therefore live on a learned submanifold along which small, structure-preserving perturbations are absorbed, while arbitrary high-magnitude directions are not. These probes are run at the GPT-2 scale on GSM8k.

3.10. Out-of-distribution generalization

To assess whether PaCT’s improvements on GSM8k reflect genuine arithmetic reasoning rather than GSM8k-specific regularities, we evaluate GSM8k-trained checkpoints zero-shot on three held-out math benchmarks: GSM-HARD, MultiArith, and SVAMP. Coconut results are obtained by rerunning the released Coconut (?) implementation under its best reported configuration for this setting, and for MultiArith we use the same evaluation subset adopted in prior continuous latent reasoning work, ensuring an apples-to-apples comparison.

As shown in Table ??, PaCT matches or exceeds Coconut across all three OOD benchmarks. This suggests that the improvements observed on GSM8k translate to related but held-out distributions rather than being tied to GSM8k-specific artifacts.

4. Limitations and Future Scope

While PaCT substantially improves the efficiency and scalability of continuous latent reasoning, it does not yet surpass language-based chain-of-thought reasoning in absolute accuracy on all tasks. Our study focuses on supervised, text-only reasoning, and extending PaCT to multimodal settings is an important direction for future work. Integrating PaCT-style latent reasoning into reinforcement learning and policy optimization frameworks such as GRPO, as well as understanding its interaction with pretraining objectives and long-horizon decision-making, may further improve the generality and robustness of latent reasoning systems also aiding in surpassing the COT upper limit otherwise more naturally present with SFT-based methods.

Exploiting the latent submanifold. Our perturbation analysis in Section ?? shows that PaCT’s latent states are tightly concentrated on a low-dimensional, structure-preserving submanifold: additive Gaussian noise is absorbed across two orders of magnitude of σ , while norm-matched random replacements — which leave the manifold

— collapse accuracy. This is a finding our perturbation experiments made visible, and it suggests an explicit modeling direction. Future methods could parameterize latent synthesis to operate directly in the learned submanifold (for instance via a low-rank or VQ-style projection, or a normalizing-flow prior over latent directions), rather than allowing the full ambient dimensionality and relying on training dynamics to discover the submanifold implicitly. Such a parameterization could improve sample efficiency, reduce redundancy across within-stage latents, and provide a more controllable interface for interpretability and intervention. This is just one direction of thought and the findings can be interpreted in multiple other ways.

From curriculum-based training to learned latent objectives at scale. The same perturbation analysis suggests a natural caveat about scaling. At the GPT-2 scale we study here, a curriculum that progressively replaces textual CoT tokens with latent reasoning stages is effective at inducing a structured latent submanifold. Prior work that has evaluated faithfulness of Coconut-style latents at much larger scales, however, has reported weaker latent-intervention effects — a pattern that we interpret not as evidence against latent reasoning per se, but as a signal that curriculum-based supervision, which effectively commits each latent to a fixed role early in training, may not be the right inductive lever for larger models with substantially more expressive latent spaces. A promising future direction is to replace the token-replacement curriculum with an explicit latent-space learning objective that (i) does not pin each latent to a single canonical role, (ii) shapes the latent submanifold directly through geometric or information-theoretic regularizers, and (iii) can be combined with the parallel stage-wise synthesis introduced in this paper. We see the submanifold structure documented here as the first concrete handle for designing such a training objective.

5. Conclusion

In this paper, we introduce Parallel Continuous Thought (PaCT), a new paradigm for continuous latent reasoning that removes the need for recurrent latent state propagation. PaCT constructs reasoning through stage-wise, parallel latent resynthesis, enabling efficient depth without incurring the serial bottlenecks of latent unrolling. Across a range of reasoning tasks and model scales, PaCT matches or improves the accuracy of prior continuous thought methods while achieving substantial gains in training and inference efficiency. Beyond performance, our analyses reveal that PaCT learns stage-specialized latent representations, exhibits more stable internal dynamics, and benefits systematically from increased reasoning depth. Together, these results challenge the assumption that effective latent reasoning must rely on propagating latent states across steps through recur-

rent unrolling, and suggest that repeated, context-grounded latent resynthesis offers a scalable alternative. We hope this work motivates further exploration of non-recurrent latent reasoning architectures for building more efficient and robust reasoning systems.

References

- Amalric, M. and Dehaene, S. A distinct cortical network for mathematical knowledge in the human brain. *NeuroImage*, 189:19–31, 2019.
- Fedorenko, E., Behr, M. K., and Kanwisher, N. Functional specificity for high-level linguistic processing in the human brain. *Proceedings of the National Academy of Sciences*, 108(39):16428–16433, 2011.
- Fedorenko, E., Piantadosi, S. T., and Gibson, E. A. Language is primarily a tool for communication rather than thought. *Nature*, 630(8017):575–586, 2024.
- Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., and Tian, Y. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- Khot, T., Trivedi, H., Finlayson, M., Fu, Y., Richardson, K., Clark, P., and Sabharwal, A. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*, 2022.
- Monti, M. M., Osherson, D. N., Martinez, M. J., and Parsons, L. M. Functional neuroanatomy of deductive inference: a language-independent distributed network. *Neuroimage*, 37(3):1005–1016, 2007.
- Monti, M. M., Parsons, L. M., and Osherson, D. N. The boundaries of language and thought in deductive inference. *Proceedings of the National Academy of Sciences*, 106(30):12554–12559, 2009.
- Monti, M. M., Parsons, L. M., and Osherson, D. N. Thought beyond language: neural dissociation of algebra and natural language. *Psychological science*, 23(8):914–922, 2012.
- Press, O., Smith, N. A., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.
- Saparov, A. and He, H. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. *arXiv preprint arXiv:2210.01240*, 2022.
- Shen, Z., Yan, H., Zhang, L., Hu, Z., Du, Y., and He, Y. Codi: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*, 2025.

- Su, D., Zhu, H., Xu, Y., Jiao, J., Tian, Y., and Zheng, Q. Token assorted: Mixing latent and text tokens for improved language model reasoning. *arXiv preprint arXiv:2502.03275*, 2025.
- Tan, W., Li, J., Ju, J., Luo, Z., Luan, J., and Song, R. Think silently, think fast: Dynamic latent compression of llm reasoning chains. *arXiv preprint arXiv:2505.16552*, 2025.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Xie, Y., Kawaguchi, K., Zhao, Y., Zhao, J. X., Kan, M.-Y., He, J., and Xie, M. Self-evaluation guided beam search for reasoning. *Advances in Neural Information Processing Systems*, 36:41618–41650, 2023.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Zelikman, E., Harik, G., Shao, Y., Jayasiri, V., Haber, N., and Goodman, N. D. Quiet-star: Language models can teach themselves to think before speaking. *arXiv preprint arXiv:2403.09629*, 2024.
- Zhang, Z., Hu, X., Zhang, H., Zhang, J., and Wan, X. Icr probe: Tracking hidden state dynamics for reliable hallucination detection in llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 17986–18002, 2025.
- Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q., et al. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.
- Zhu, H., Hao, S., Hu, Z., Jiao, J., Russell, S., and Tian, Y. Reasoning by superposition: A theoretical perspective on chain of continuous thought. *arXiv preprint arXiv:2505.12514*, 2025.

A. Stability and initialization details

To stabilize optimization, we ground latent reasoning states in the task specification. Let h_q denote the hidden state of the final question token. We initialize the output projection of the latent synthesis cross-attention to zero and add h_q as a residual to each latent state after synthesis. This ensures that, at initialization, all latent states correspond to grounded copies of the question representation and only deviate when learning discovers useful refinements.

This grounding serves two purposes: (i) it anchors latent reasoning to the task context throughout training, and (ii) it prevents unstructured accumulation of latent history by ensuring that each stage depends only on the current textual context and a fixed task representation.

B. Attention masking and causal structure

We enforce the following attention constraints:

1. Text tokens attend causally to earlier text tokens.
2. Text tokens do not attend to future latent states.
3. Latent states attend to all previous text tokens and to other latent states within the same stage bidirectionally.
4. Answer tokens attend to all prior text tokens and all latent states from previous stages.

This realizes a $Text \rightarrow Latents \rightarrow Text$ pattern while preserving autoregressive decoding and preventing label leakage.

C. PaCT training algorithm

Algorithm ?? provides the full curriculum training procedure for PaCT.

Algorithm 1 Curriculum training for PaCT

Require: Dataset $\mathcal{D}$, curriculum schedule $\{m_s, S_s\}$, slots L , parameters θ , stepsize η

- 1: **for** $s = 0, \dots, S_{\max}$ **do**
 - 2: **for** $(q, r, a) \sim \mathcal{D}$ **do**
 - 3: Replace first m_s textual CoT tokens in r with S_s latent stages
 - 4: $x \leftarrow [q; \tilde{r}_s; a]$
 - 5: Compute $p_\theta(\cdot | x)$ and latent states $Z^{(1:S_s)}$
 - 6: Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$
 - 7: **end for**
 - 8: **end for**
-

D. Ablations on Latent Tokens

This section studies how PaCT utilizes latent tokens, with a focus on latent capacity and redundancy. In particular, we analyze (i) whether PaCT can outperform recurrent latent reasoning using fewer latent tokens, and (ii) whether multiple latent tokens within the same stage encode complementary or redundant information at inference time.

D.1. Single-Latent PaCT vs. Dual-Latent Coconut

We first examine the minimum latent capacity required for effective continuous latent reasoning. Coconut relies on recurrent latent unrolling, where multiple latent tokens are generated sequentially and each latent must carry forward information from previous steps. In contrast, PaCT synthesizes latent tokens in parallel from the current context, without propagating latent states across steps. To isolate the effect of latent capacity, we compare PaCT using a single latent token per stage ($c_{\text{thought}} = 1$) against Coconut using two recurrently unrolled latent tokens ($c_{\text{thought}} = 2$), following standard Coconut settings on GSM8k. As shown in Table ??, PaCT with only one latent token already outperforms Coconut (33.20% vs. 32.00%).

This result indicates that PaCT does not rely on increased latent width to achieve strong performance. Instead, its stage-wise, context-grounded latent resynthesis produces more information-dense latent representations, allowing PaCT to match or exceed the performance of recurrent latent reasoning while using fewer latent tokens.

Table 9. Single-latent PaCT vs. dual-latent Coconut on GSM8k.

Model	c_{thought}	Accuracy (%)
PaCT	1	33.20
Coconut	2	32.00

D.2. Latent Redundancy Within Stages

We investigate whether multiple latent tokens within the same PaCT stage encode complementary information or exhibit redundancy. We consider a PaCT model trained with two latent tokens per stage and three stages ($c_{\text{thought}} = 2$, $S = 3$), resulting in six latent tokens in total. At inference time, we selectively retain only one latent token per stage, discarding the other.

Table 10. Accuracy when keeping only 1 of 2 latents per stage (PaCT trained with $c_{\text{thought}} = 2$, $S = 3$ on GSM8k).

Configuration	# Latents	Accuracy (%)
Full model	6	34.4
First of each stage (L0, L2, L4)	3	31.6
Second of each stage (L1, L3, L5)	3	31.6

Table ?? reports the resulting accuracy. Retaining either the first or the second latent token from each stage yields identical performance (31.6%), regardless of which latent is kept. Although accuracy decreases relative to the full model (34.4%), the symmetry across configurations indicates that latent tokens within the same stage encode largely redundant information.

This observation is consistent with the high within-stage similarity measured in our latent similarity analysis and supports the design choice of allocating compute toward greater depth (more stages) rather than increased latent width. In PaCT, reasoning benefits more from additional stages than from duplicating latent capacity within a stage.

E. Latent decoding analysis – extension

This section provides additional qualitative evidence to complement the latent decoding analysis in Sec. ?. Our goal is not to claim that latent reasoning states correspond to explicit natural language tokens, but rather to probe whether latent trajectories encode meaningful intermediate computations that align with the ground-truth reasoning process.

Algorithm 2 Top- k Latent Decoding Probe for PaCT/Coconut ($k=5$)

Require: Transformer f_θ with latent stages, output matrix $W_{\text{out}} \in \mathbb{R}^{|\mathcal{V}| \times d}$, input x , stages S

Ensure: Top- k trajectories $\{(V_s, P_s)\}_{s=1}^S$

- 1: Compute latent embeddings $\{z_s \in \mathbb{R}^d\}_{s=1}^S \leftarrow f_\theta(x)$
 - 2: **for** $s = 1, \dots, S$ **do**
 - 3: $\ell_s \leftarrow W_{\text{out}} z_s$
 - 4: $p_s \leftarrow \text{softmax}(\ell_s)$
 - 5: $V_s \leftarrow \text{top-}k(p_s)$, $P_s \leftarrow \{p_s(v) \mid v \in V_s\}$
 - 6: **end for**
 - 7: **return** $\{(V_s, P_s)\}_{s=1}^S$
-

We provide additional decoded latent trajectories to supplement Sec. ?.

Top- k latent decoding probe. We apply the Top- k Latent Decoding Probe (Alg. ??) to both PaCT and Coconut models. For each latent state, we project the latent embedding into vocabulary space using the output projection matrix and inspect the top- k tokens along with their probabilities. This probe serves as a diagnostic tool to visualize how internal latent representations evolve across reasoning stages and whether they track intermediate quantities relevant to the final answer.

Progressive refinement in PaCT. Across correct examples (Examples S1–S2), PaCT exhibits a clear stage-wise progression from intermediate quantities toward the final answer. Early latent stages often decode to coarse intermediate values

(e.g., partial sums or products), while later stages increasingly concentrate probability mass on the correct final answer. Importantly, latent pairs within the same stage frequently decode to identical tokens with similar probabilities, reflecting the high within-stage similarity reported in Table 4 of the main paper. This behavior is consistent with PaCT’s design: latent states within a stage act as parallel hypotheses derived from the same contextual grounding, rather than representing sequential reasoning steps.

Example S1**Correct**

Question: Maggie went to Lou’s aquarium and saw 100 goldfish. She was allowed to catch half, and she caught $\frac{3}{5}$ of that. How many remain to catch?

Ground Truth CoT: $100 \times \frac{1}{2} = 50 \rightarrow 50 \times \frac{3}{5} = 30 \rightarrow 50 - 30 = 20$

Predicted: 20 **GT:** 20

	L0	L1	L2	L3	L4	L5
Token	50	50	30	30	20	20
Prob	13%	14%	13%	20%	55%	56%

Interpretation: Intermediate-to-final progression with sharpening confidence.

Example S2**Correct**

Question: Mitchell has 8 packets of gum, 7 pieces each, and chews all but 2. How many does he chew?

Ground Truth CoT: $8 \times 7 = 56 \rightarrow 56 - 2 = 54$

Predicted: 54 **GT:** 54

	L0	L1	L2	L3	L4	L5
Token	56	56	54	54	54	54
Prob	5.2%	6.5%	25%	50%	87%	83%

Interpretation: Answer emerges early and confidence sharpens across stages.

Example S3**Incorrect**

Question: Roman and Remy used 33 gallons. Remy used 1 more than $3 \times$ Roman. How many did Remy use?

Ground Truth CoT: $x + (3x + 1) = 33 \rightarrow x = 8 \rightarrow 3x + 1 = 25$

Predicted: 26.25 **GT:** 25

	L0	L1	L2	L3	L4	L5
Token	34	34	9	9	25	25
Prob	11%	12%	7.5%	8.6%	22%	26%

Interpretation: Latents reach the correct answer late, but decoding still commits to an incorrect output.

Example S4**Incorrect**

Question: Suzy starts with 98 books. 43 checked out. Next day 23 returned and 5 checked out. Friday 7 returned. How many books?

Ground Truth CoT: $98 - 43 = 55 \rightarrow 55 + 23 = 78 \rightarrow 78 - 5 = 73 \rightarrow 73 + 7 = 80$

Predicted: 119 **GT:** 80

	L0	L1	L2	L3	L4	L5
Token	69	69	76	72	70	70
Prob	3.6%	3.4%	0.9%	1.6%	11%	9.3%

Interpretation: Multi-step tracking fails; low probabilities reflect uncertainty.

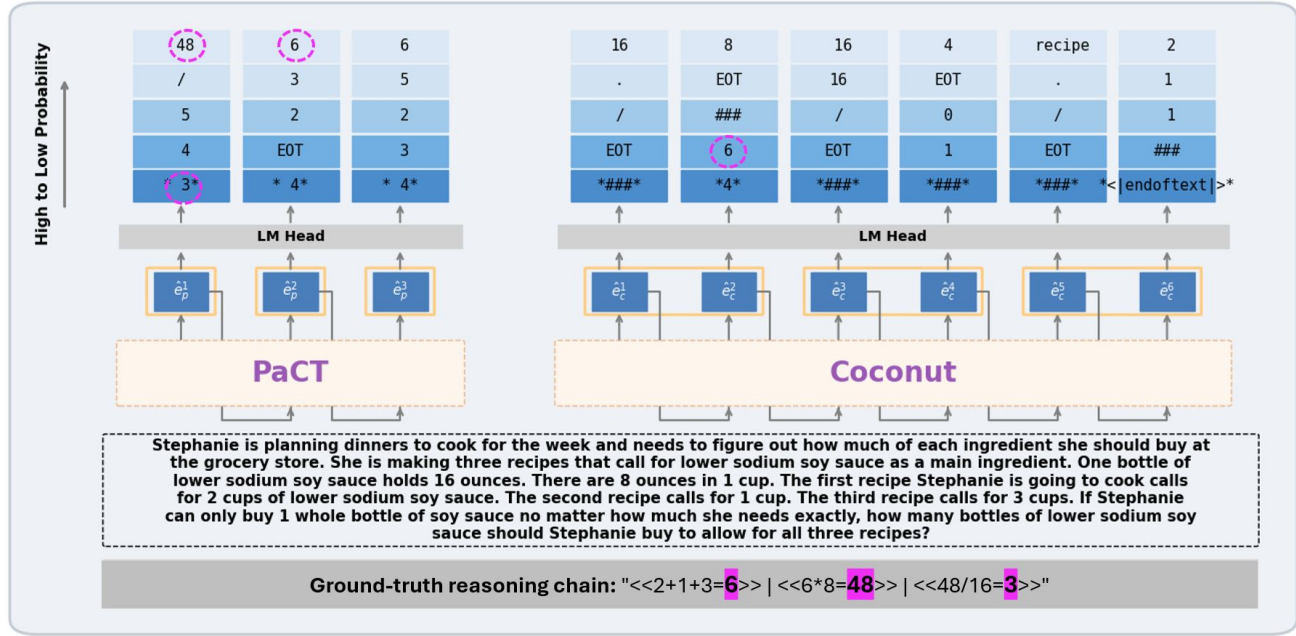


Figure 9. PaCT vs Coconut latent decoding comparison - example 1

Example S5**Incorrect****Question:** Opera tickets increased from \$85 to \$102. What is the percent increase?**Ground Truth CoT:** $102 - 85 = 17 \rightarrow \frac{17}{85} \times 100 = 20\%$ **Predicted:** 12 **GT:** 20

	L0	L1	L2	L3	L4	L5
Token	73	73	.	.	12	12
Prob	3.5%	3.1%	12%	18%	25%	24%

Interpretation: Reasoning collapses; the model commits to an incorrect answer without computing the correct percentage.

Failure modes and uncertainty signals. In incorrect cases (Examples S3–S5), the latent decoding probe reveals informative failure patterns. In some instances, PaCT latents reach the correct intermediate or even final value at later stages but fail to sustain sufficient confidence for correct decoding (Example S3). In others, the latent trajectory captures partial intermediates but fails to correctly compose them (Example S4), or collapses early to an incorrect estimate without completing the required computation (Example S5). These cases are characterized by low or diffuse probability mass across decoded tokens, indicating uncertainty rather than confident hallucination.

Comparison with Coconut. In contrast to PaCT, Coconut’s recurrently unrolled latent states exhibit less structured progression. Decoded tokens often fluctuate across stages, mix intermediate and irrelevant quantities, or prematurely terminate with end-of-text or non-numeric tokens. This behavior is consistent with Coconut’s recurrent latent propagation, where each latent must simultaneously carry forward prior information and perform new computation, leading to entangled representations and higher cross-stage similarity. As a result, Coconut latent trajectories are less interpretable and less aligned with the underlying arithmetic structure of the task.

Together, these extended examples reinforce three key observations. First, PaCT latent states encode meaningful intermediate quantities despite never being supervised to produce text. Second, reasoning in PaCT emerges as a stage-wise refinement process rather than a single evolving latent trajectory. Third, decoding probabilities provide a useful confidence signal: correct predictions are associated with sharpening probability mass on the final answer, while failures exhibit diffuse or unstable latent confidence. These findings support the claim that PaCT’s parallel, stage-wise latent resynthesis yields more structured and interpretable internal reasoning dynamics than recurrent latent unrolling.

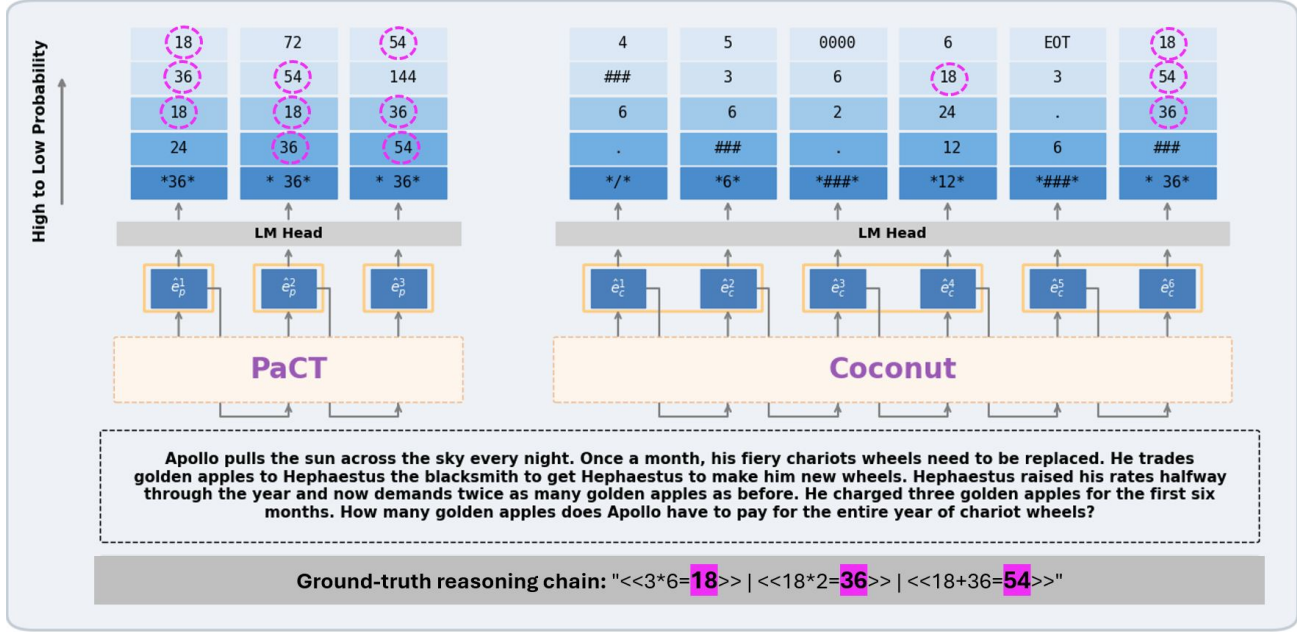


Figure 10. PaCT vs Coconut latent decoding comparison - example 2

F. Inference latency

Figure ?? reports inference time. PaCT is consistently faster than Coconut, though the relative gain is smaller than for training because autoregressive answer decoding remains a dominant cost. This is an expected behavior as PaCT primarily removes serial dependence during latent generation; it does not change the causal decoding interface for answer tokens.

G. Source Code and Model Availability

To support reproducibility and facilitate future research, we release the full source code and trained model weights for PaCT. The repository includes training scripts, evaluation pipelines, and configuration files required to reproduce all experiments reported in this paper.

Our implementation and pretrained models are publicly available at:

<https://anonymous.4open.science/r/pact-576C/>



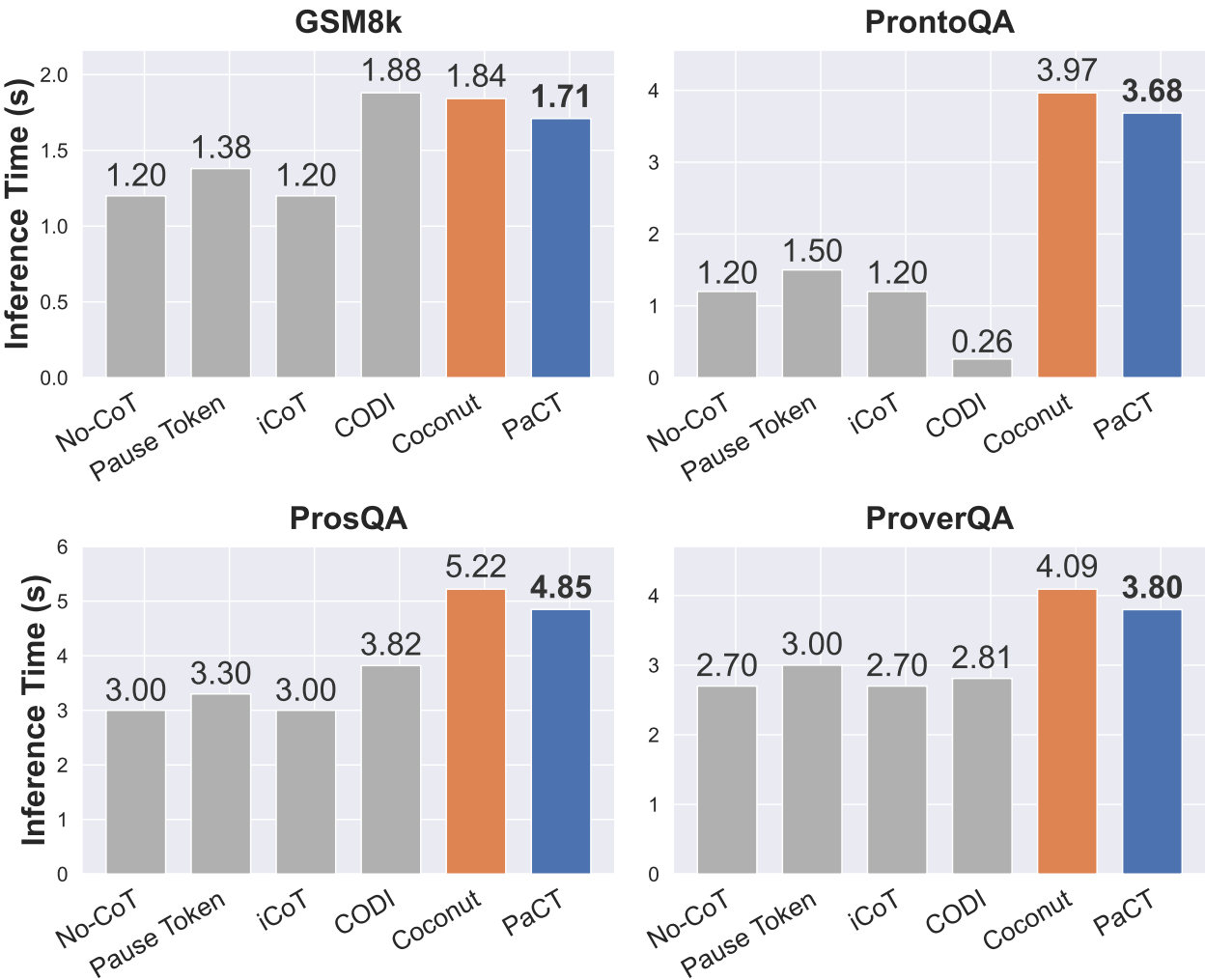


Figure 12. **Inference time (seconds).** Inference time (per sample) for different reasoning methods across four benchmarks. PaCT reduces latency relative to Coconut by removing recurrent latent-unrolling overhead, while preserving competitive performance.